

PostgreSQL Collation Tutorial

Collation in PostgreSQL is a set of rules that defines how text strings are compared and sorted. In other words, a collation determines the ordering of characters (for example, whether “a” comes before or after “A”) and how equality (`=`, `<`, `>`, etc.) works on text. PostgreSQL allows you to specify a collation at the database, table-column, or even expression level, overriding the database’s default locale-based order ¹. By default, a database inherits the OS locale settings (`LC_COLLATE` and `LC_CTYPE`), but explicit collations can be applied using `COLLATE`. In PostgreSQL 15+ there are three providers of collations: the built-in **C** (code-point) collation (`provider = c`), the **libc** (OS locale) collation (`provider = libc`), and the **ICU** (Unicode Collation Algorithm) collation (`provider = icu`) ³ ⁴.

1. How Collation Affects Sorting and Comparison

Collation rules affect all string comparisons and ordering operations. Whenever you compare text or sort it (for example in `ORDER BY`, or using operators like `<`, `>`, `=`, `ILIKE`, etc.), PostgreSQL uses the collation of the string(s) involved to decide the result ⁵ ⁶. For instance, **under the simple C collation (which sorts by raw Unicode code points), uppercase letters come before lowercase** (`'Z' < 'a'`), and characters are compared by their byte values. **In contrast, a locale collation like `en_US.UTF-8` (managed by the OS or ICU) implements language-specific rules: often case is ignored or considered last, accented characters may compare as equal to their base letter, and number ordering or punctuation might behave differently.** For example, under the `POSIX` or `C` collation “A” sorts before “a”, but under many locale collations “a” may sort before “A” (case-insensitive ordering) ⁷ ⁸.

Example: Create a table and compare ordering under different collations.

```
CREATE TABLE t (a text);
INSERT INTO t VALUES ('a'),('A'),('ii'),('II'),('*'),('1.1'),('-');
```

- Default (locale-based) sort might yield: `*, -, 1.1, a, A, ii, II`.
- Using `COLLATE "C"` (or `POSIX`) yields: `*, -, 1.1, A, II, a, ii` (uppercase before lowercase) ⁷.
- Using a language-specific ICU collation changes results (e.g., `COLLATE "zh-Hans-x-icu"` reorders `-` before `*`) ⁹. In general, **collation differences can dramatically change sort order and equality** ⁷ ⁶.

2. Default vs Custom Collations

- **Default Collations:** Each database has a default collation (set by `initdb` using `LC_COLLATE` / `LC_CTYPE`), and each column inherits it unless overridden. The default is usually an OS locale (like `en_US.UTF-8`) or the simple C collation (`"C"` or `"POSIX"`). These built-in collations exist in the system catalog (e.g. `"C"` and `"POSIX"` always exist) ¹⁰.

- **Custom Collations:** You can create your own collations via `CREATE COLLATION` ¹¹ ¹². A custom collation can be a copy of an existing one or use ICU to tweak rules. For example, you might create an accent-insensitive or case-insensitive collation. The main difference is that custom collations let you choose a provider (`libc` or `icu`), locale, and flags (like `deterministic = false` for case-insensitivity). For instance:

```
-- Create a German collation based on OS locale (libc)
CREATE COLLATION german (provider = libc, locale = 'de_DE.utf8');

-- Create an ICU-based collation (language tag style)
CREATE COLLATION french (provider = icu, locale = 'fr-FR');
```

Custom collations allow **tailoring string order** beyond the defaults (e.g. ignoring accents or changing letter order). Unlike the fixed database default, they can be created any time and applied to columns or queries. However, custom collations must exist in the database to be used, and their behavior depends on the underlying locale data ¹¹ ¹².

3. Using Collations in SQL: `ORDER BY`, `LIKE`, Comparisons

You can apply a collation directly in SQL using `COLLATE "name"`. This overrides the default collation for that expression. Examples:

- **Ordering:**

```
-- Use German collation for ordering
SELECT * FROM t ORDER BY a COLLATE "de_DE.utf8";
```

This forces German sorting rules (e.g. treating "ä" after "a", etc.).

The Redrock tutorial shows how different collations affect `ORDER BY` ⁷. For instance:

```
SELECT * FROM t ORDER BY a COLLATE "POSIX";
```

gave uppercase letters first, whereas the default (English ICU) did not ⁷.

- **Comparison Operators:**

Collation also affects `=` or `<`. For example, if you have a case-insensitive collation:

```
CREATE COLLATION case_insensitive (provider = icu, locale = 'und-u-ks-
level2', deterministic = false);
SELECT 'Z' = 'z' COLLATE case_insensitive; -- returns true
```

If you mix collations incorrectly, Postgres may error. E.g. if `a` has collation "de_DE" and `b` has "es_ES", then `SELECT a < b` is ambiguous and errors; you must specify one explicitly ¹³:

```
SELECT a < b COLLATE "de_DE"; -- resolves the conflict
```

- **Pattern Matching (LIKE, ILIKE):**

Pattern matching also uses collation rules for character classification. For example, a binary collation like "C" will treat % and _ as in ASCII, whereas a locale collation might have different rules for case-folding or accent sensitivity. In practice, using collations in LIKE is less common. Instead, use ILIKE for case-insensitive or define a case-insensitive collation. For example:

```
-- Default behavior (case-sensitive)
SELECT 'Straße' LIKE 'Strasse'; -- false in most collations (ß ≠ ss)

-- Using an accent-insensitive collation (example with ICU)
CREATE COLLATION ign_case_acc (provider=icu, locale='und-u-ks-level1',
deterministic=false);
SELECT 'Stra\u00DFe' = 'Strasse' COLLATE ign_case_acc; -- true (ß equals
ss, if collation defines it)
```

(The actual result depends on the collation rules; ICU collations can be customized for such comparisons.)

In summary, adding COLLATE "name" after a string or identifier changes how PostgreSQL compares or sorts that expression^{13 7}. It works with ORDER BY, comparisons (=, <, >), and also affects operators like ILIKE which rely on character properties.

4. Listing Available Collations

To see which collations you can use, query the system catalog **pg_collation** or use psql meta-commands. For example:

```
SELECT * FROM pg_collation;
```

will show all collations recognized by your PostgreSQL instance¹⁰. In psql, you can also use \dOS+ to list OS (and ICU) collations. The initial set of collations in a database is populated from the operating system locales at initdb time¹⁴. The pg_collation table (or \dOS+) shows the collation name, provider, locale, encoding, etc.^{15 10}. For example: - "C" and "POSIX" (built-in) always appear. - Names like en_US.utf8, de_DE.utf8 (libc) or und-x-icu (ICU root) may appear. Using SELECT * on pg_collation is the general way to list all collations available in the current database¹⁰.

5. Creating and Using a Custom Collation

You can create a custom collation with the `CREATE COLLATION` command. This lets you pick the provider (`libc` or `icu`), the locale (OS locale or ICU language tag), and options like `deterministic`. For example:

```
-- 1. Create a case-insensitive (but accent-sensitive) collation using ICU:
CREATE COLLATION case_insensitive (
  provider = icu,
  locale = 'und-u-ks-level2',
  deterministic = false
);

-- 2. Create an accent-insensitive collation:
CREATE COLLATION ignore_accents (
  provider = icu,
  locale = 'und-u-ks-level1-kc-true',
  deterministic = false
);

-- Test custom collation:
SELECT 'Å' = 'A' COLLATE ignore_accents; -- returns true (ignores accent)
SELECT 'z' < 'A' COLLATE case_insensitive; -- returns false (case-insensitive)
```

In the example above, `ignore_accents` treats “Å” (A with ring) equal to “A” ¹⁶. The `case_insensitive` collation makes `'z' = 'Z'` and `'B' < 'b'` true ¹⁷. You must then use these collations explicitly, e.g. `SELECT * FROM table ORDER BY name COLLATE ignore_accents`. Alternatively, you can assign a custom collation as the default for a column:

```
CREATE TABLE users (
  name TEXT COLLATE case_insensitive
);
```

This makes all comparisons on `users.name` use the custom collation by default. Remember to create custom collations in a schema that will be preserved by backups (often a non-`pg_catalog` schema) ¹⁸.

6. Collation, Indexing, and Performance

Collation choice can affect index use and query performance. B-tree indexes on text use the column's collation to order the index entries. This means:

- **Index stability:** A collation must be *immutable* for an index to remain valid. If the underlying collation rules change (for example, due to an OS or ICU upgrade), the index's ordering could

become incorrect. PostgreSQL tracks collation versions: if a collation's version on disk doesn't match the system's current version, you'll see a warning like:

```
WARNING: collation "xx-x-icu" has version mismatch ... A change in
collation definitions can lead to corrupt indexes... all objects
depending on the collation should be rebuilt (e.g. REINDEX) and then use
ALTER COLLATION ... REFRESH VERSION 19 20 .
```

In practice, after upgrading the OS or ICU, you should `REINDEX` affected indexes and run `ALTER COLLATION ... REFRESH VERSION` to clear warnings ²⁰ .

- **Index usage with predicates:** For prefix or range queries, note that a C (binary) collation is straightforward (index can be used for `LIKE 'prefix%'`), because it is simple byte order. Other collations may have complex rules: an index on a column with a non-C collation can usually still be used, but certain expressions or case variations might not be index-friendly (e.g. an ICU collation that reorders characters could make a prefix search fall outside index scans).
- **Performance:** Simple binary (`"C"`) collation is fastest because it just compares bytes. Language-aware collations incur extra overhead. In benchmarks, using `COLLATE "C"` often improves sort and scan speed by a significant margin ²¹ ⁸ . As Adarsh Rajesh notes: “the 'C' collation... can be more performant, as indexing is based purely on raw byte values rather than language-specific rules” ⁸ . In a recent test, switching from `en_US` (glibc) to `C` reduced query times by ~20-50% in some cases ²¹ . However, you trade human-friendly ordering for speed. Use `"C"` for internal identifiers or slugs if appropriate.

7. Considerations for Multilingual Applications

Supporting multiple languages can complicate collation choice. **No single collation handles all scripts optimally** ²² . For example, a German collation might treat “ä” as “ae”, whereas a Spanish collation sorts “ñ” after “n”. If your data mix languages (English, Chinese, Arabic, etc.), you must choose carefully:

- PostgreSQL relies on OS or ICU collations. You might pick a **neutral ICU collation** like `und-x-icu` (the “root” or language-agnostic sort) for a generic ordering ²³ .
- Alternatively, handle different languages separately: one table column per language, or add an explicit `COLLATE` clause in queries per user locale.
- Be aware that applying, say, an English collation to Chinese text yields meaningless order. In many cases, it's acceptable for non-primary languages to just use a default Unicode order.
- ICU collations offer some customization (like `emoji` sort, numeric sort, etc.) which might help specific needs ²⁴ .
- If sorting multi-language text is crucial, you may need to normalize or tag text by language. Often, full-text search (with language dictionaries) is used instead of relying solely on sort order.

In short, for multilingual apps ensure your chosen collation(s) make sense for your data's languages, and test sorting of representative strings. Remember Laurenz Albe's caution: “there will never be a perfect way to collate strings across languages” and mixing languages in one column can yield unpredictable order ²² .

8. Common Pitfalls and Troubleshooting

- **Unexpected Sort Results:** If "ORDER BY" yields surprising order, check the active collation. Use `COLLATE` explicitly or create separate columns per language if needed. Remember that uppercase vs lowercase ordering, accent handling, and special characters all depend on collation ⁷ ⁸.
- **Collation Conflicts:** Mixing collations in one expression can cause errors. For example, comparing two columns with different implicit collations without `COLLATE` leads to an error ²⁵. The fix is to explicitly `COLLATE` one side so both use the same rules.
- **Collation Name Variants:** On some OSes, multiple names refer to the same locale (e.g. `de_DE` and `de_DE.utf8`). PostgreSQL often creates both. Avoid mixing stripped (`de_DE`) and full names (`de_DE.utf8`) as they are distinct objects ²⁶, even if behavior is identical. Similarly, `"C"` and `"POSIX"` act the same but are distinct in PostgreSQL.
- **Case-/Accent-insensitive Comparison:** If you expect `'A' = 'a'` to be true, use `ILIKE` or a case-insensitive collation, since ordinary `=` is case-sensitive under most collations.
- **Index Corruption After OS Upgrade:** As mentioned, updating glibc or ICU can change sort rules. If you see a warning about collation version mismatch, you should `REINDEX` and `ALTER COLLATION ... REFRESH VERSION` ²⁰.
- **Missing Collation Support:** Custom collations using `provider = icu` require PostgreSQL built with ICU support. If `SELECT * FROM pg_collation` shows no `x-icu` names, ICU isn't available.
- **Locale Installation:** To use a given `libc` locale, that locale must exist on the OS (`locale -a`). If `initdb` didn't find it, you may need to install the OS locale or use `CREATE COLLATION FROM`.

By following these guidelines, explicitly setting appropriate collations, and testing your queries, you can avoid common collation bugs and ensure that string comparisons and sorting behave as intended.

Sources: Official PostgreSQL documentation and recent articles/blogs on PostgreSQL collations ² ¹³ ⁸ ¹⁰ ⁷ ²⁰, which provide the definitions, SQL examples, and nuanced behavior described above.

¹ ³ ⁸ Understanding Collations in PostgreSQL | by Adarsh Rajesh | Medium

<https://medium.com/@adarsh2801/understanding-collations-in-postgresql-648e4fa333e1>

² ⁴ ⁵ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁶ ¹⁷ ¹⁸ ²³ ²⁵ ²⁶ PostgreSQL: Documentation: 17: 23.2. Collation Support

<https://www.postgresql.org/docs/current/collation.html>

⁶ Manage collation changes in PostgreSQL on Amazon Aurora and Amazon RDS | AWS Database Blog

<https://aws.amazon.com/blogs/database/manage-collation-changes-in-postgresql-on-amazon-aurora-and-amazon-rds/>

⁷ ⁹ Collations and string sort order in PostgreSQL - Redrock Postgres

<https://www.rockdata.net/blog/collations-and-sort-order/>

¹⁵ PostgreSQL: Documentation: 17: 51.12. pg_collation

<https://www.postgresql.org/docs/current/catalog-pg-collation.html>

¹⁹ ²⁰ postgresql - Collation version mismatch - Database Administrators Stack Exchange

<https://dba.stackexchange.com/questions/324649/collation-version-mismatch>

²¹ How much speed you're leaving at the table if you use default locale? – select * from depez;

<https://www.depez.com/2024/06/11/how-much-speed-youre-leaving-at-the-table-if-you-use-default-locale/>

22 **unicode - Multiple languages with utf8 in postgresql - Stack Overflow**
<https://stackoverflow.com/questions/39461851/multiple-languages-with-utf8-in-postgresql>

24 **Overview of ICU collation settings | Peter Eisentraut**
<http://peter.eisentraut.org/blog/2023/05/16/overview-of-icu-collation-settings>